SDD UNIVERSAL

v1.2.0

Manual de uso para personas no técnicas

Paso a paso, desde cero y sin asumir experiencia en programación

Objetivo del manual Que puedas instalar SDD, configurarlo y usarlo en un proyecto real entendiendo qué hace cada parte, qué mensajes son normales y cuándo realmente tenés que intervenir.

Versión cubierta SDD Universal v1.2.0, publicada como release estable en GitHub el 28/10/2026. npm no forma parte de esta distribución.

Repositorio oficial: https://github.com/Ikkora/ai-sdd-starter

Release: https://github.com/Ikkora/ai-sdd-starter/releases/tag/v1.2.0

Cómo usar este manual

No necesitás leerlo entero antes de empezar. Si es tu primera vez, seguí los capítulos 1 a 8 en orden. Después podés volver a las secciones de consulta cuando aparezca un mensaje o una duda concreta.

Si querés...	Andá a...
Entender qué es SDD sin tecnicismos	Capítulo 1
Empezar lo más fácil posible	Capítulos 2 y 3
Crear un proyecto desde GitHub	Capítulo 4
Instalar SDD dentro de un proyecto existente	Capítulo 5
Decirle a SDD qué tecnologías usa tu proyecto	Capítulo 6
Configurar seguridad	Capítulo 7
Hacer tu primer pedido al agente	Capítulo 9
Entender FAST, DEBUG, STANDARD y STRICT	Capítulo 10
Retomar luego de un corte o límite de uso	Capítulo 12
Resolver errores comunes	Capítulo 18
Tener un machete rápido	Capítulo 20

Regla de oro SDD está pensado para que vos describas el objetivo en lenguaje natural. No necesitás dirigir cada archivo ni cada comando. Tu trabajo es explicar qué querés y responder solamente cuando exista una decisión real.

1. ¿Qué es SDD Universal?

SDD significa Spec-Driven Development. En palabras simples: en vez de pedirle a un agente de IA “hacé esto” y dejar que improvise, SDD le da un proceso ordenado para entender el pedido, planificarlo, dividirlo en tareas, revisar riesgos, validar el resultado y dejar evidencia de lo que hizo.

Pensalo como un encargado de obra digital. Vos decís qué querés construir. SDD organiza al equipo, controla el avance y evita que cada sesión empiece desde cero.

Tu pedido	Plan	Tareas	Construye	Pruebas	Revisión	Cierre

1.1. ¿Qué problema intenta resolver?

- Que un agente olvide lo que ya hizo después de un corte de sesión.
- Que cambie muchas cosas de golpe sin plan ni trazabilidad.
- Que diga “terminado” sin pruebas suficientes.
- Que un cambio pequeño de seguridad sea tratado como un cambio trivial.
- Que la metodología quede atada a PHP, Python, JavaScript u otro lenguaje.
- Que las revisiones se repitan una y otra vez sin criterio.

1.2. Lo que SDD no es

- No es un lenguaje de programación.
- No es un editor de código.
- No reemplaza Git ni GitHub.
- No es un antivirus ni un escáner SAST.
- No garantiza que un producto sea invulnerable.
- No obliga a usar un stack particular.

Importante La Security Baseline organiza controles y evidencia de seguridad. Deep Review busca defectos técnicos concretos en los cambios. El Reviewer final decide si la spec quedó correctamente cumplida. Son funciones distintas.

2. Qué necesitás antes de empezar

2.1. Camino recomendado para principiantes

La forma más simple es usar el repositorio oficial como GitHub Template. Para ese camino no necesitás instalar el CLI de SDD ni Node.js solamente para crear la estructura inicial.

☐ Una cuenta de GitHub.

☐ Acceso a un agente de programación que pueda trabajar con los archivos de tu repositorio.

☐ Git instalado si vas a clonar el proyecto en tu computadora.

☐ Una carpeta donde guardar el proyecto.

☐ Una idea clara, aunque sea breve, de qué querés construir o modificar.

2.2. Si vas a usar el CLI

El CLI oficial de SDD v1.2.0 necesita Node.js 18 o superior. La distribución no está publicada en npm, por lo que el CLI se ejecuta desde una copia local de la distribución.

No hace falta npm global La release oficial se distribuye por GitHub. No busques “npm install sdd-v2” como paso obligatorio, porque v1.2.0 no fue publicada en npm.

3. Elegir cómo instalar SDD

Opción	Ideal para	Ventaja	Dificultad
A. GitHub Template	Proyecto nuevo	Es la más directa. Copiás una estructura lista.	Baja
B. CLI sdd init	Proyecto nuevo o existente	Instala la estructura SDD dentro de una carpeta o repo.	Media

Mi recomendación Si nunca usaste SDD, empezá con GitHub Template. Cuando entiendas la estructura, el CLI resulta mucho más fácil de usar.

4. Instalación fácil: GitHub Template

Este es el recorrido recomendado para una persona sin experiencia técnica.

Paso 1. Abrir el repositorio oficial

1. Entrá a: https://github.com/Ikkora/ai-sdd-starter
2. Verificá que el repositorio sea Ikkora/ai-sdd-starter.
3. Si querés comprobar la versión estable, abrí Releases y buscá SDD Universal v1.2.0.

Paso 2. Crear tu propio proyecto

4. Presioná el botón “Use this template” en GitHub.
5. Elegí crear un nuevo repositorio.
6. Escribí el nombre de tu proyecto. Por ejemplo: mi-sistema-turnos.
7. Elegí si querés el repositorio público o privado.
8. Confirmá la creación.

Qué acaba de pasar GitHub creó una copia independiente del starter. Tu nuevo repositorio tiene su propia historia futura y puede adaptar la configuración a tu producto.

Paso 3. Clonar el proyecto en tu PC

Si tu agente trabaja directamente en GitHub, este paso puede variar. Si vas a trabajar localmente, copiá la URL del repositorio y clonalo.

```
git clone <URL-DE-TU-REPOSITORIO>
cd <NOMBRE-DE-TU-REPOSITORIO>
```

Paso 4. Revisar los archivos principales

Archivo o carpeta	Para qué sirve
AGENTS.md	Reglas generales que debe respetar el agente.
.sdd/project.yaml	Configuración específica de tu proyecto.
docs/constitution.md	Principios que no deberían romperse.
.agents/skills/sdd/	Protocolo de funcionamiento de SDD.
specs/	Lugar donde se guarda la historia de cada cambio importante.

No edites el motor porque sí Para adaptar tu proyecto, normalmente vas a tocar .sdd/project.yaml, AGENTS.md y docs/constitution.md. El motor genérico debe seguir siendo genérico.

5. Instalación alternativa: CLI sdd init

Este camino sirve si ya tenés una carpeta o repositorio y querés incorporar SDD sin empezar desde el template.

Paso 1. Tener una copia local de SDD Universal

Cloná el repositorio oficial o descargá la release v1.2.0. En este ejemplo suponemos que lo clonaste en una carpeta separada.

```
git clone https://github.com/Ikkora/ai-sdd-starter.git
cd ai-sdd-starter
git checkout v1.2.0
```

Paso 2. Ir al proyecto donde querés instalar SDD

Abrí una terminal en la raíz de tu proyecto. “Raíz” significa la carpeta principal, donde normalmente está el .git, package.json, composer.json, README o archivos equivalentes.

Paso 3. Hacer primero una simulación

Antes de escribir archivos, usá --dry-run. Esto muestra el plan de instalación sin modificar nada.

```
node <RUTA-A-SDD>/cli/sdd.js init --dry-run
```

Muy recomendado Si el proyecto ya tiene archivos, usá siempre --dry-run primero. Es la forma más tranquila de ver si habrá conflictos.

Paso 4. Instalar

```
node <RUTA-A-SDD>/cli/sdd.js init --name "Mi proyecto"
```

Una instalación nueva crea, entre otros elementos, stack vacío, capabilities vacías y security.profile en standard.

Paso 5. Si hay conflictos

Sin --force, SDD detecta los conflictos antes de escribir y aborta antes de la primera modificación. Eso es una protección.

--force sólo puede reemplazar destinos administrados por SDD. Aun así, no lo uses a ciegas porque podrías reemplazar una configuración SDD que vos ya personalizaste.

```
node <RUTA-A-SDD>/cli/sdd.js init --force
```

Cuidado con --force No es un “arreglar todo”. Es una autorización para reemplazar archivos SDD administrados. Primero entendé el conflicto.

6. Configurar tu proyecto sin ser programador

El archivo central es .sdd/project.yaml. YAML es simplemente una forma de guardar información con sangrías. No necesitás programar para editarlo, pero sí respetar espacios y estructura.

6.1. La parte más importante: project

```
project:
  name: "Mi proyecto"
  constitution: "docs/constitution.md"
  specs_root: specs
  memory: null
```

name es el nombre visible. constitution apunta al archivo con principios del proyecto. specs_root indica dónde se guardan las specs. memory puede quedar en null si no usás una memoria operativa adicional.

6.2. Declarar el stack

Stack significa “conjunto de tecnologías”. SDD v1.2.0 no usa una lista cerrada, así que podés declarar tecnologías conocidas, futuras o propias.

Ejemplo PHP + JavaScript + MySQL

```
stack:
  backend:
    language: php
    version: "8.3"
  frontend:
    language: javascript
  database:
    engine: mysql
```

Ejemplo Python

```
stack:
  backend:
    language: python
    framework: fastapi
  database:
    engine: postgresql
```

Ejemplo con una tecnología no conocida por SDD

```
stack:
  api_custom:
    language: future-lang
    framework: custom-framework
```

Eso no genera error. Si no existe un ruleset específico, SDD conserva la baseline universal de seguridad.

6.3. No inventar datos por completar

Podés dejarlo vacío Si todavía no sabés el stack, stack: {} es válido. Es mejor declarar poco y correcto que rellenar tecnologías por intuición.

7. Configurar seguridad

SDD v1.2.0 incorpora Security Profiles. El perfil no “enciende un antivirus”; cambia qué controles y exigencia de revisión se aplican dentro del proceso SDD.

Perfil	Qué hace	Cuándo usarlo
off	Desactiva controles y elevación específicos de la Security Baseline.	Casos deliberados donde no querés la capa SDD de seguridad.
standard	Aplica controles core y contextuales de forma proporcional.	Recomendado para la mayoría de proyectos.
strict	Agrega controles extendidos, reduce umbrales de review y exige justificar excepciones.	Sistemas sensibles o cambios de alto riesgo.

```
security:
  profile: standard
```

Para empezar Dejá standard salvo que tengas una razón clara para cambiarlo. Es el valor que reciben las instalaciones nuevas.

7.1. Capabilities: qué puede hacer tu sistema

Capabilities son características del producto que ayudan a SDD a entender contexto. No elevan el riesgo por sí solas: la elevación necesita que el cambio actual toque una superficie relacionada.

```
capabilities:
  authentication: true
  authorization: true
  personal_data: true
  file_uploads: false
  payments: false
  external_apis: true
```

7.2. Ejemplos sencillos

Tu sistema...	Capability sugerida
Tiene inicio de sesión	authentication: true
Tiene roles/permisos	authorization: true
Guarda nombre, email, DNI u otros datos personales	personal_data: true
Permite subir PDFs o imágenes	file_uploads: true

Tu sistema...	Capability sugerida
Procesa cobros	payments: true
Se conecta con APIs de terceros	external_apis: true

La idea clave Una capability describe al producto. Un tag de cambio describe lo que estás modificando ahora. SDD sólo recomienda elevar el modo cuando el cambio realmente toca una superficie de riesgo relevante.

8. Personalizar las reglas de tu proyecto

8.1. docs/constitution.md

La constitución guarda principios que el agente no debería romper. Es un buen lugar para reglas de negocio, seguridad, compatibilidad y calidad.

Ejemplos escritos en lenguaje común:

- “Ningún usuario puede ver información de otro cliente salvo que tenga permiso explícito.”
- “Nunca se debe eliminar una venta confirmada sin dejar trazabilidad.”
- “Las migraciones de base de datos deben tener rollback o una estrategia de recuperación.”
- “No guardar contraseñas ni tokens en el repositorio.”

8.2. AGENTS.md

AGENTS.md contiene reglas del repositorio y fuentes de verdad. No conviene copiar todo el motor ahí. El template oficial deja espacios para arquitectura, seguridad, invariantes y Definition of Done propios del proyecto.

Menos es más Poné reglas que de verdad cambien decisiones. Un archivo lleno de frases genéricas se vuelve ruido y dificulta que el agente identifique lo importante.

9. Tu primer uso de SDD

La entrada puede ser sdd_run(goal), pero no necesitás escribir una función especial. Una petición normal en lenguaje natural activa el mismo flujo si el agente está trabajando con las reglas SDD del repositorio.

Ejemplo 1. Una funcionalidad nueva

Prompt recomendado Copiá y pegá este texto en el chat del agente.

> Quiero agregar recuperación de contraseña por email. El usuario debe poder solicitar un enlace temporal, usarlo una sola vez y definir una contraseña nueva. Usá SDD y respetá las reglas actuales del proyecto.

Ejemplo 2. Un error

Prompt recomendado Copiá y pegá este texto en el chat del agente.

> Hay un error: cuando edito el teléfono de un cliente se guarda correctamente, pero al volver a abrir la ficha aparece el valor anterior. Investigalo y corregilo usando SDD.

Ejemplo 3. Un cambio pequeño

Prompt recomendado Copiá y pegá este texto en el chat del agente.

> Cambiar el texto del botón “Guardar registro” por “Guardar”. Usá SDD y mantené el resto sin cambios.

Qué no hace falta decir

- “Abrí este archivo, después este otro, después corré tal test”, salvo que vos realmente necesites imponer ese procedimiento.
- “Pedime permiso antes de cada paso”. SDD está diseñado para autoavanzar dentro del alcance aprobado.
- “Hacé un commit después de cada archivo”. Los commits se hacen por bloques lógicos revisados.

10. Entender FAST, DEBUG, STANDARD y STRICT

El modo determina cuánta ceremonia, planificación y revisión necesita el cambio. El agente lo clasifica según el trabajo y el riesgo.

Modo	En palabras simples	Ejemplo
FAST	Cambio trivial, local y reversible.	Cambiar un texto o nombre sin afectar contratos.
DEBUG	Hay un defecto y primero hay que encontrar la causa.	Un formulario guarda mal un valor.
STANDARD	Feature o refactor normal.	Agregar un filtro o una pantalla.

Modo	En palabras simples	Ejemplo
STRICT	Riesgo elevado o categoría sensible.	Permisos, datos personales, pagos, migraciones destructivas.

No elijas STRICT para todo Más estricto no siempre significa mejor. SDD busca validación proporcional. Si todo fuera STRICT, el proceso perdería eficiencia y el riesgo dejaría de distinguirse.

11. Qué pasa después de enviar un pedido

Carga reglas	Busca pendiente	Clasifica	Planifica	Ejecuta	Valida	Revisa	Cierra

11.1. RESUME FIRST

Antes de empezar algo nuevo, SDD busca una spec incompleta. Si existe, intenta continuar exactamente desde resume.next_action. Esta es una de las piezas más importantes del sistema.

11.2. Spec, plan y tareas

Artefacto	Pregunta que responde
spec.md	¿Qué queremos lograr y por qué?
plan.md	¿Cómo se va a implementar?
tasks.md	¿Qué unidades concretas de trabajo hay?
validation.md	¿Qué evidencia demuestra que funciona?
state.yaml	¿En qué punto exacto estamos y qué sigue?

11.3. Checkpoints

En STANDARD los checkpoints típicos están alrededor del 50% y 100%. En STRICT, la configuración inicial usa 25%, 50%, 75% y 100%. El objetivo no es frenar por deporte: es revisar el incremento en momentos razonables.

12. Qué hacer si se corta la sesión o se alcanza un límite

No empieces de cero. Si state.yaml quedó bien persistido, SDD sabe qué tarea o acción corresponde después.

Prompt para reanudar Copiá y pegá este texto en el chat del agente.

```
RESUME FIRST. Continuá desde el estado persistido de la spec activa. No repitas tareas, tests o reviews ya registrados como completos. Verificá Git y ejecutá exactamente resume.next_action.
```

No limpies el repositorio Si SDD registró el árbol como dirty_expected, esos cambios son parte del trabajo. No uses reset, clean o checkout para “dejar todo prolijo” salvo que exista una decisión consciente y segura.

12.1. Qué significa next_action

Ejemplo	Significado
execute:T04	Ejecutar la tarea T04.
validate_checkpoint:50	Validar el checkpoint del 50%.
run_quality_gates:final	Ejecutar Quality Gates finales.
review_block:api	Revisar el bloque llamado api.
commit_block:api	Crear el commit lógico de ese bloque.
await_human_decision:...	Hace falta una decisión tuya.
await_human_setup:...	Falta preparar algo externo.
await_human_repair:...	Hay un entorno externo que requiere reparación humana.

13. Cuándo tenés que intervenir y cuándo no

Sí deberías intervenir cuando...

- Hay dos opciones funcionales válidas y sólo vos podés decidir cuál quiere el producto.
- Hay que aceptar un riesgo conocido.
- Faltan credenciales o configuración externa que el agente no debe crear.
- Apareció una operación destructiva o irreversible.

- El alcance cambió respecto del pedido original.
- Hay más de una spec activa y no está claro cuál continuar.

No hace falta intervenir para...

- Autorizar cada test.
- Decidir si pasa a la siguiente tarea ya aprobada.
- Autorizar un review previsto por la política.
- Autorizar un commit lógico ya previsto.
- Repetir información que ya está persistida en la spec.

14. Quality Gates sin jerga

Quality Gates son controles adicionales que pueden ejecutarse según el tipo de cambio y la configuración.

14.1. Anti-Slop

Se usa de forma contextual para detectar salida genérica, copy inflado, patrones repetitivos, complejidad accidental y otros problemas de calidad cuando el artefacto lo justifica. No es un scanner de seguridad.

14.2. Deep Review

Es una revisión técnica de sólo lectura enfocada en defectos concretos del cambio: correctitud, regresiones, seguridad, mantenibilidad, complejidad y límites arquitectónicos.

14.3. SDD Reviewer

Es la autoridad final del proceso: comprueba cumplimiento de la spec, arquitectura, evidencia, excepciones y cierre.

Componente	Pregunta principal
Security Baseline	¿Qué controles deberían aplicar en este contexto?
Anti-Slop	¿La salida tiene problemas de calidad o complejidad accidental?
Deep Review	¿El cambio introdujo defectos técnicos concretos?
SDD Reviewer	¿La spec quedó realmente cumplida y puede cerrarse?

14.4. off, auto y required en Quality Gates

Modo	Comportamiento
off	No se usa.
auto	Se usa si aplica y está disponible. Si falta, se registra y el proceso puede continuar.
required	Si aplica y falta, SDD debe esperar setup humano.

15. Usar sdd doctor

doctor revisa si la infraestructura y configuración de SDD están saludables. No escanea el código de tu producto buscando vulnerabilidades.

```
node <RUTA-A-SDD>/cli/sdd.js doctor
```

En v1.2.0 separa cuatro superficies: SDD Core, Project Configuration, Security Policy y Quality Gates.

Cómo interpretar el resultado

Resultado	Qué significa
HEALTHY	La instalación y configuración obligatoria están correctas.
NOT DETECTED en algo opcional	SDD no pudo confirmar esa integración. No necesariamente está mal.
Un provider required no detectado	Falta setup obligatorio y la instalación no se considera saludable.
Error de configuración	Revisá .sdd/project.yaml o los manifests que indique el mensaje.

Doctor no revisa tu app No usa PHP, Python, MySQL ni servicios externos para diagnosticar el producto. Revisa SDD.

16. Comandos del CLI explicados

16.1. Ver versión

```
node <RUTA-A-SDD>/cli/sdd.js --version
```

Para v1.2.0 debería mostrar 1.2.0.

16.2. Inicializar

```
node <RUTA-A-SDD>/cli/sdd.js init
```

16.3. Inicializar con nombre

```
node <RUTA-A-SDD>/cli/sdd.js init --name "Mi proyecto"
```

16.4. Declarar stack desde el comando

```
node <RUTA-A-SDD>/cli/sdd.js init --stack backend.language=php --stack database.engine=mysql
```

16.5. Declarar capabilities

```
node <RUTA-A-SDD>/cli/sdd.js init --capability authentication=true --capability personal_data=true
```

16.6. Elegir perfil de seguridad

```
node <RUTA-A-SDD>/cli/sdd.js init --security strict
```

16.7. Combinar opciones

```
node <RUTA-A-SDD>/cli/sdd.js init --name "Portal Clientes" --stack backend.language=php --stack frontend.language=javascript --stack database.engine=mysql --capability authentication=true --capability personal_data=true --security standard --dry-run
```

17. Ejemplo completo: agregar login a un sistema PHP

Supongamos que tenés un sistema PHP + JavaScript + MySQL y querés incorporar login.

Paso 1. Configurar contexto

```
stack:
  backend:
    language: php
  frontend:
    language: javascript
  database:
    engine: mysql

capabilities:
  authentication: true
  authorization: true
  personal_data: true

security:
  profile: standard
```

Paso 2. Explicar el objetivo

Pedido inicial Copiá y pegá este texto en el chat del agente.

```
Quiero agregar login al sistema. Los usuarios deben ingresar con email y contraseña. Necesito cierre de sesión y protección de las pantallas privadas. No quiero modificar otras funciones. Usá SDD.
```

Paso 3. Dejar que SDD clasifique

Como authentication y authorization son superficies sensibles, SDD puede recomendar un tratamiento más exigente según el cambio real. Vos no necesitás forzar manualmente el modo si el proceso ya lo justifica.

Paso 4. Leer el primer checkpoint

Revisá que el objetivo coincida con lo que pediste, que no aparezca alcance inventado y que la arquitectura propuesta no cambie cosas ajenas.

Paso 5. Responder sólo si hace falta

Si el agente pregunta, por ejemplo, “¿las cuentas las crea un administrador o el usuario se registra solo?”, esa sí es una decisión funcional válida. Respondé eso y dejá que continúe.

Paso 6. Cierre

Al 100%, buscá evidencia de tests, Quality Gates aplicables, Reviewer final aprobado y working tree en el estado esperado. “Parece funcionar” no reemplaza la evidencia registrada.

18. Problemas comunes y qué hacer

Situación	Qué hacer
“Hay conflictos durante sdd init”	Volvé a ejecutar con --dry-run, leé qué archivos chocan y decidí si querés preservar o reemplazar los archivos SDD.
“Quiero usar --force”	Usalo sólo si entendés qué archivo SDD va a reemplazar. No es un botón de reparación universal.
“El agente quiere empezar de cero después de un corte”	Pedí RESUME FIRST y que lea state.yaml antes de hacer trabajo nuevo.
“Aparece await_human_decision”	Respondé la decisión concreta. No hace falta rediseñar el proceso.
“Aparece await_human_setup”	Falta una integración/configuración externa requerida. Completá ese setup o cambiá conscientemente la política.
“Doctor dice NOT DETECTED para algo opcional”	No asumas que todo está roto. Revisá si el provider está en auto/off o required.
“Mi lenguaje no está reconocido”	No es un error. La baseline universal sigue aplicando aunque no haya ruleset específico.
“El cambio parece demasiado revisado”	Comprobá que no se haya configurado strict o required sin necesidad y que la elevación tenga una superficie de riesgo real.
“Quiero instalar Anti-Slop automáticamente con SDD”	SDD no instala terceros silenciosamente. Anti-Slop sigue siendo una integración externa opcional.
“Quiero publicar npm”	v1.2.0 se distribuye oficialmente por GitHub. npm no forma parte de este release.

19. Reglas simples para usar SDD con seguridad

- ☐ No pegues contraseñas, tokens o claves privadas dentro de archivos versionados.
- ☐ No uses git reset --hard, git clean o checkout para borrar cambios sólo porque el árbol está dirty.
- ☐ No aceptes una operación destructiva si no entendés su efecto.
- ☐ No marques todo como STRICT por costumbre.
- ☐ No uses off para “hacer que pase” un cambio sensible.

☐ No confundas Doctor con un escáner de vulnerabilidades.

☐ No conviertas AGENTS.md en una copia del motor.

☐ No inventes stack o capabilities sólo para rellenar campos.

☐ Antes de --force, hacé --dry-run.

☐ Antes de cerrar una tarea importante, buscá evidencia, no sólo una frase de éxito.

20. Machete rápido

Quiero…	Hago esto
Empezar un proyecto nuevo	Usar GitHub Template.
Instalar en un repo existente	sdd init, primero con --dry-run.
Decir qué tecnología uso	Editar stack en .sdd/project.yaml o usar --stack.
Declarar login/datos/uploads/etc.	Editar capabilities o usar --capability.
Seguridad normal	security.profile: standard.
Seguridad más exigente	security.profile: strict, si existe una razón real.
Pedir una feature	Explicarla en lenguaje natural y decir “Usá SDD”.
Reportar un bug	Describir qué pasa, qué esperabas y pedir investigación con SDD.
Retomar después de un corte	“RESUME FIRST. Continuá desde state.yaml”.
Ver salud de SDD	sdd doctor.
Ver versión	sdd --version.
Simular instalación	sdd init --dry-run.

Prompt universal recomendado

Plantilla Copiá y pegá este texto en el chat del agente.

```
Quiero [OBJETIVO].

Contexto importante:
- [REGLA O RESTRICCIÓN 1]
- [REGLA O RESTRICCIÓN 2]

Resultado esperado:
- [QUÉ DEBE QUEDAR FUNCIONANDO]

Usá SDD. Reanudá trabajo previo si existe y detenete sólo ante una decisión humana real, un bloqueo comprobado o un cambio de alcance.
```

21. Glosario sin tecnicismos

Término	Significado simple
Baseline	Conjunto base de controles que sirve como piso común.
Capability	Algo que el producto puede hacer, como login, pagos o uploads.
Checkpoint	Punto de control del avance.
CLI	Programa que se usa escribiendo comandos en una terminal.
Commit	Registro de un bloque de cambios en Git.
Deep Review	Revisión técnica profunda de los cambios.
Diff	Diferencia entre una versión y otra.
Doctor	Diagnóstico de la instalación/configuración de SDD.
Legacy	Proyecto creado con una versión anterior que debe conservar compatibilidad.
Manifest	Archivo declarativo que describe reglas o componentes.
Quality Gate	Control adicional que puede ejecutarse según contexto.
RESUME FIRST	Regla que obliga a continuar trabajo persistido antes de iniciar algo nuevo.

Término	Significado simple
Reviewer	Agente independiente que valida cumplimiento y cierre.
Ruleset	Conjunto de reglas específico para cierto stack o contexto.
SAST	Análisis estático automatizado de seguridad. La baseline SDD no es un SAST.
Spec	Documento que define qué se quiere lograr y cómo se comprueba.
Stack	Tecnologías del proyecto.
State	Estado persistido que dice exactamente en qué punto está el trabajo.
Working tree	Archivos actuales del repositorio, incluyendo cambios todavía no commiteados.

22. Referencias oficiales usadas para este manual

Este manual se preparó contra la distribución oficial SDD Universal v1.2.0. Para cuestiones normativas o detalles técnicos, las siguientes fuentes del repositorio tienen prioridad:

- README.md - instalación, comandos y conceptos públicos.
- PLAYBOOK.md - operación diaria.
- engine/skill/references/engine.md - contrato metodológico detallado.
- engine/skill/references/quality-gates.md - Quality Gates y providers.
- templates/install/.sdd/project.yaml - configuración inicial instalada.
- templates/install/AGENTS.md - reglas iniciales del repositorio.
- templates/install/docs/constitution.md - constitución inicial del proyecto.
- GitHub Release SDD Universal v1.2.0 - publicación estable oficial.

Nota de publicación La release v1.2.0 está publicada y estable en GitHub. Algunos textos versionados dentro del tag fueron escritos antes del acto final de publicación y todavía describen v1.2.0 como “preparada”. Para el estado público actual, prevalece la release oficial de GitHub.

23. La idea más importante para llevarte

SDD no pretende que una persona no técnica se convierta en especialista en Git, seguridad o arquitectura. Pretende que el agente trabaje con un proceso trazable y que vos puedas intervenir en el nivel correcto: objetivos, reglas del producto y decisiones reales.

Si dudás entre “decirle exactamente cómo programarlo” o “explicar claramente qué necesitás y qué no puede romperse”, elegí lo segundo. El motor existe justamente para organizar el cómo.

En una frase Describí bien el objetivo, declarale a SDD el contexto de tu proyecto y dejá que el proceso haga el trabajo pesado, pero exigí evidencia antes de aceptar el cierre.